

SPI Slave with Register Map

1. Protocol Overview

The design is a synthesizable 4-wire SPI slave (`spi_slave.v`) connected to a 32768 x 8-bit register map (`register_map.v`) through a top-level wrapper (`spi_top.v`). It operates in SPI Mode 0, most significant bit first, and supports single-byte and burst read/write access with auto-incrementing addresses.

Signal	Direction	Description
<code>sclk</code>	Input	SPI clock, Mode 0 (CPOL = 0, CPHA = 0). Also used as the register map's memory clock.
<code>csb</code>	Input	Chip select, active low. HIGH resets the FSM to IDLE and tri-states <code>sdo</code> .
<code>sdi</code>	Input	Serial data in (MOSI). Carries the header and write data, MSB first.
<code>sdo</code>	Output	Serial data out (MISO), tri-state. Driven only during DATA_READ.

1.1. Header Format

Every transaction begins with a 16-bit header shifted in on `sdi`: bit 15 selects read (1) or write (0), and bits 14:0 give the starting 15-bit address. There is no dedicated burst-mode bit. burst is simply a transaction where `csb` is held low for more than one data byte after the header.

2. Module Architecture & Design Details

2.1. SPI Slave FSM (`spi_slave.v`)

The slave is controlled by a 2-bit state register clocked on the rising edge of `sclk`, with `csb` as an asynchronous reset back to IDLE.

State	Encoding	Behavior
IDLE	2'b00	Held while <code>csb</code> is high. On the first rising edge after <code>csb</code> drops, samples <code>sdi</code> as the R/W bit and moves to HEADER.
HEADER	2'b01	Shifts in the 15-bit address, one bit per rising edge, into the <code>addr</code> register. After the 15th address bit, jumps to DATA_READ or DATA_WRITE based on the stored R/W bit.
DATA_WRITE	2'b10	Shifts in 8 bits of write data. On the cycle after the last bit, pulses <code>wr_en</code> for one clock. On the following cycle, <code>wr_en</code> is cleared and <code>addr</code> is incremented, allowing burst writes to continue seamlessly.
DATA_READ	2'b11	Shifts out 8 bits of <code>rd_data</code> via the SDO drive logic. Increments <code>addr</code> after each byte, supporting burst reads.

Read Turnaround (SDO Drive Logic)

SDO is driven by a separate always block clocked on the falling edge of `sclk`. While the FSM is in DATA_READ, this block latches `rd_data[bit_cnt]` into `sdo_reg` and asserts `sdo_oe`; otherwise `sdo_oe` is deasserted and the `sdo` output goes to high-impedance (1'bz) via the tri-state assign statement. Driving on the falling edge, combined with the register map's combinational read path, is what lets the slave present the first read bit within half a clock cycle of the last address bit being sampled.

Burst Handling in DATA_WRITE / DATA_READ

Both data states use a 4-bit `bit_cnt` that reloads to 7 after each byte. In DATA_WRITE, `wr_en` is pulsed for one cycle once a full byte has been shifted in, and `addr` is incremented on the following cycle when `wr_en` is seen high, so back-to-back bytes are written correctly without gaps. In DATA_READ, `addr` is simply incremented once `bit_cnt` wraps, so the next byte's data is ready in `rd_data` in time for the next falling edge.

2.2. Register Map (register_map.v)

A 32768 x 8-bit memory array, initialized to zero at time 0 for simulation.

- **Combinational read:** rd_data is continuously assigned from memory[addr], which is required to meet the half-clock read turnaround — there is no clocked read latency to account for.
- **Synchronous write:** writes occur on the negative edge of clk (which is selk, wired through spi_top) when wr_en is high. Writing on the falling edge guarantees the final bit of a byte is captured before csb can rise.

2.3. Top-Level Wrapper (spi_top.v)

spi_top.v simply instantiates spi_slave and register_map and wires them together: selk drives both the SPI FSM and the memory clock, and the addr / wr_data / wr_en / rd_data signals connect the two modules directly with no arbitration or decoding logic, since there is only one slave device.

3. Testbench Verification (tb_spi_slave.v)

The testbench is self-checking: it drives the DUT through a set of reusable SPI master tasks (spi_send_bit, spi_send_header, spi_write_byte, spi_read_byte_check) and cross-checks every read against a reference memory array (ref_mem), incrementing an error counter on any mismatch. Errors and transaction counts are summarized at the end of the run.

Test 1: Single-byte write

- **Action:** Write 0xAB to address 15'h0010.
- **Checks:** The DUT memory at 15'h0010 is compared directly against the expected value via hierarchical reference to u_spi_top.u_register_map.memory.

Test 2: Single-byte read

- **Action:** Read back address 15'h0010.
- **Checks:** spi_read_byte_check compares the returned byte against 0xAB and also verifies the SDO turnaround timing bit-by-bit during the read.

Test 3: Burst write

- **Action:** Write four consecutive bytes (0xC0-0xC3) starting at 15'h0100 in a single transaction.
- **Checks:** Each of the four memory locations is compared against the expected value after csb is raised.

Test 4: Burst read

- **Action:** Read back the same four bytes written in Test 3 as a single burst.
- **Checks:** Each byte returned is checked against ref_mem, verifying auto-increment and continuous SDO output across byte boundaries.

Test 5: Random writes and read-back (100 transactions)

- **Action:** 100 iterations of a randomized address and data byte, each written and then immediately read back as two separate transactions.
- **Checks:** Every read-back is checked against the value just written, exercising arbitrary addresses and payloads across the full memory range.

Test 6: Random burst transfers

- **Action:** An 8-byte burst write of random data starting at 15'h2000, followed by an 8-byte burst read of the same region.
- **Checks:** Each of the 8 bytes read back is compared against ref_mem.

Test 7: CSB interruption

- **Action:** A write header is sent to address 15'h1234, but csb is raised partway through the data byte (after 4 of 8 bits), aborting the transaction.
- **Checks:** Verifies that the partially-shifted byte (0xA0) was never committed to memory, since wr_en only pulses after all 8 bits have been received.

Test 8: Address rollover

- **Action:** A 3-byte burst write starting at 15'h7FFE, which crosses the top of the address space (15'h7FFF) and wraps to 15'h0000 (since addr is a 15-bit register).
- **Checks:** Reads back all three bytes and confirms the wraparound values match, verifying correct address rollover behavior.

Test 9: Back-to-back transactions

- **Action:** Two single-byte write transactions issued with `csb` toggling high then immediately low again, with no idle gap.
- **Checks:** Confirms the FSM correctly resets and restarts on the next `csb` assertion with no corruption of the first transaction's data.

Test 10: Read turnaround timing

Not a separate transaction — the testbench notes that the half-clock SDO turnaround is already exercised on every read inside `spi_read_byte_check` (used in Tests 2, 4, 5, 6, and 8), which explicitly checks that `sdo` matches the expected bit partway through the low phase of `sclk`, before the master's sampling edge.

3.1. Pass/Fail Reporting

At the end of the run, the testbench prints the total transaction count and total error count, and reports ">>> ALL TESTS PASSED <<<" if no errors were recorded, or ">>> TEST FAILED <<<" otherwise. A 5,000,000 ns watchdog timer is included to catch a hung simulation.

3.4. Results

```
=====
#   STARTING SPI SLAVE VERIFICATION
=====
# Test 1: Single-byte write
# Test 2: Single-byte read
# Test 3: Burst write
# Test 4: Burst read
# Test 5: Random writes + compare (100 txns)
# Test 6: Random burst transfers
# Test 7: CSB interruption
# Test 8: Address rollover
# Test 9: Back-to-back transactions
# Test 10: Read turnaround timing (Verified natively in Test 2/4/5/8)
=====
# Transactions: 208
# Errors:      0
# >>> ALL TESTS PASSED <<<
=====
# ** Note: $finish      : tb spi_slave.v(245)
```

